🧭 Navigation2 Single Point Autonomous Navigation

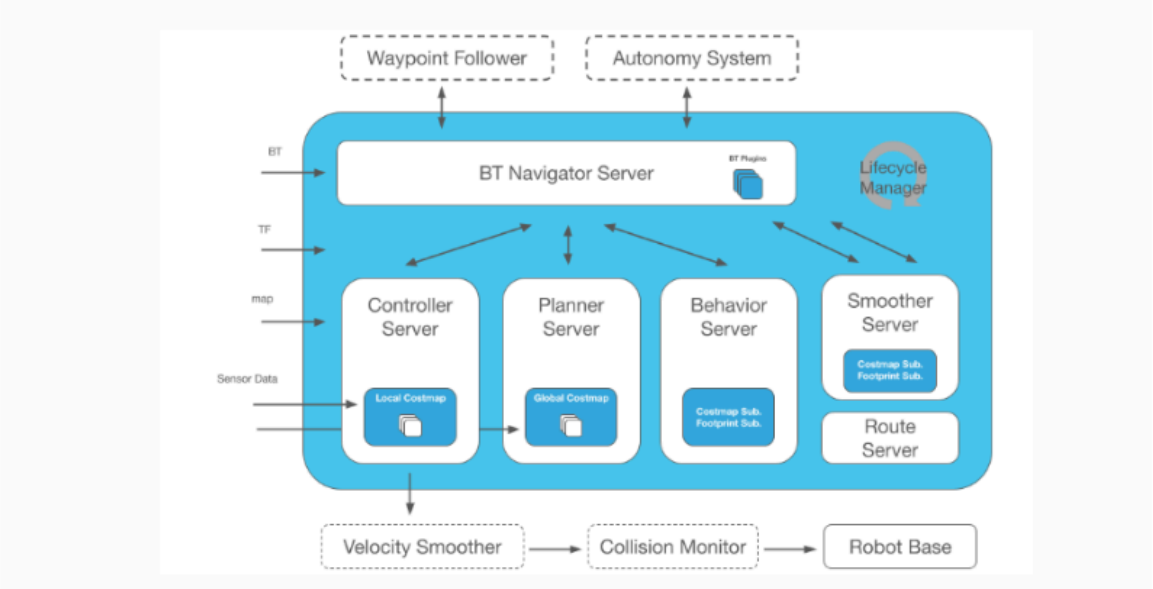
1. Course Content

- Master using ROS 2 official Navigation2 (Nav2) navigation framework combined with TEB local path planning algorithm for autonomous single-point navigation in known maps.
- Master AMCL particle filter initial pose setting, global path planning, TEB local dynamic obstacle avoidance trajectory generation, and precise destination arrival control.

---

2. Navigation2 Framework and Navigation Principle

Navigation2 (Nav2) is a modular autonomous mobile robot navigation system in ROS 2:



Nav2 core works collaboratively with the following modules:

1. **Map Server**: Loads offline-built OccupancyGrid occupancy raster map, providing global static obstacle layer.
2. **Global Localization (AMCL)**: Adaptive Monte Carlo particle filter localization, uses lidar point cloud matching with map contours to estimate robot's absolute pose in global map in real-time and publishes `map -> odom` TF transformation.
3. **Costmap 2D**:
    - **Global Costmap**: Built based on static map, used by global planner to search for optimal path.
    - **Local Costmap**: Real-time rolling update of dynamically detected obstacles centered on robot, with safety distance inflation (Inflation).
4. **Global Planner (Nav2 Planner)**: Uses Dijkstra / A* algorithm to compute collision-free global path from start to goal on global costmap.
5. **Local Planner (TEB Local Planner)**: Based on Time-Elastic-Band algorithm, under Ackermann chassis

kinematics and velocity/acceleration constraints, real-time optimizes smooth local obstacle avoidance trajectory and velocity control commands (`/cmd_vel`).

6. **Behavior Tree Navigator (BT Navigator)**: Unified scheduling of planning, tracking, obstacle avoidance retries, and recovery behaviors through behavior tree.

---

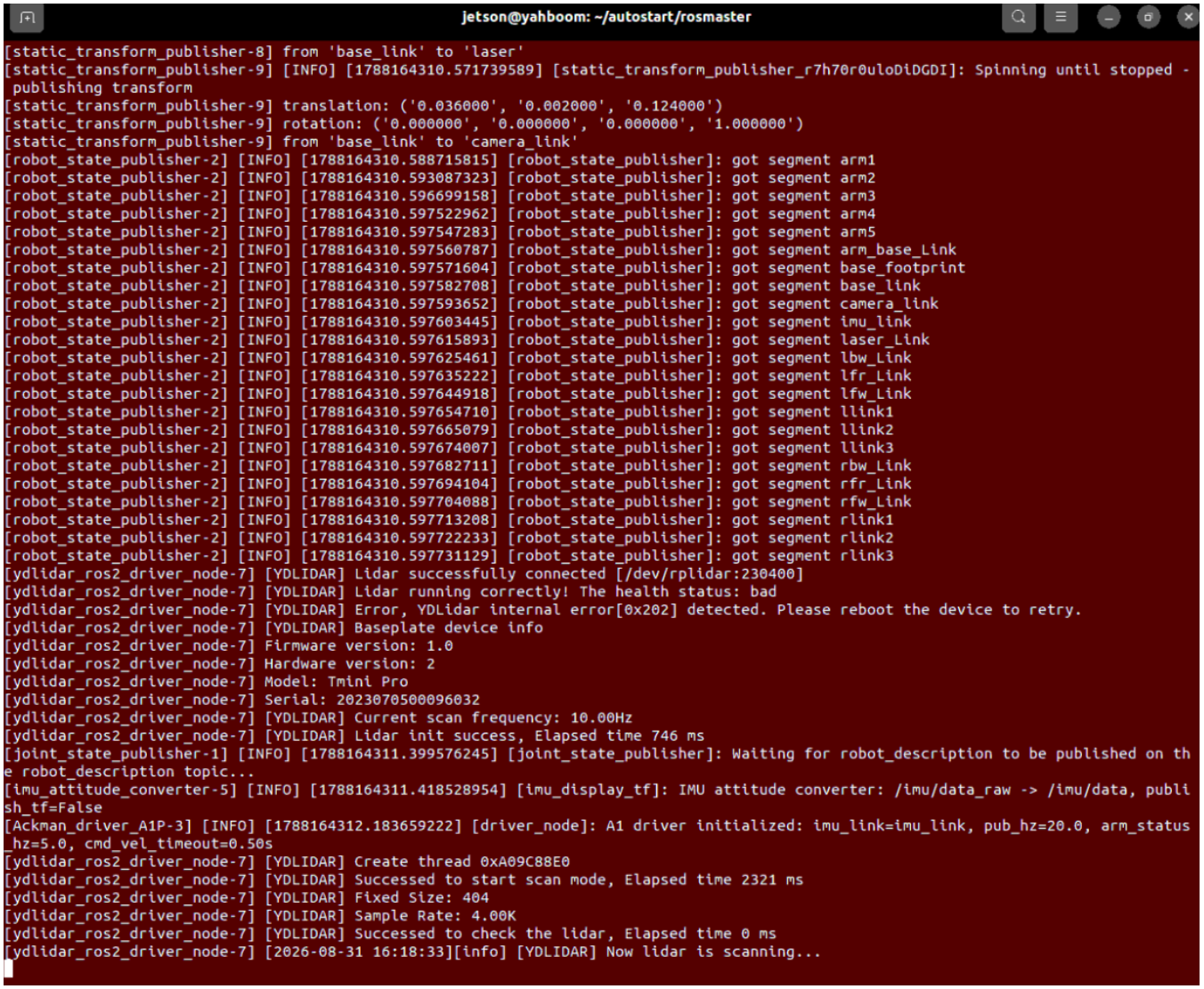
3. Preparation and Running

This example requires opening 3 terminals to start the following nodes in sequence:

3.1 Start Chassis + Lidar

Run in **Terminal 1** (one-click start of A1 Pro chassis driver, IMU attitude solution, and lidar):

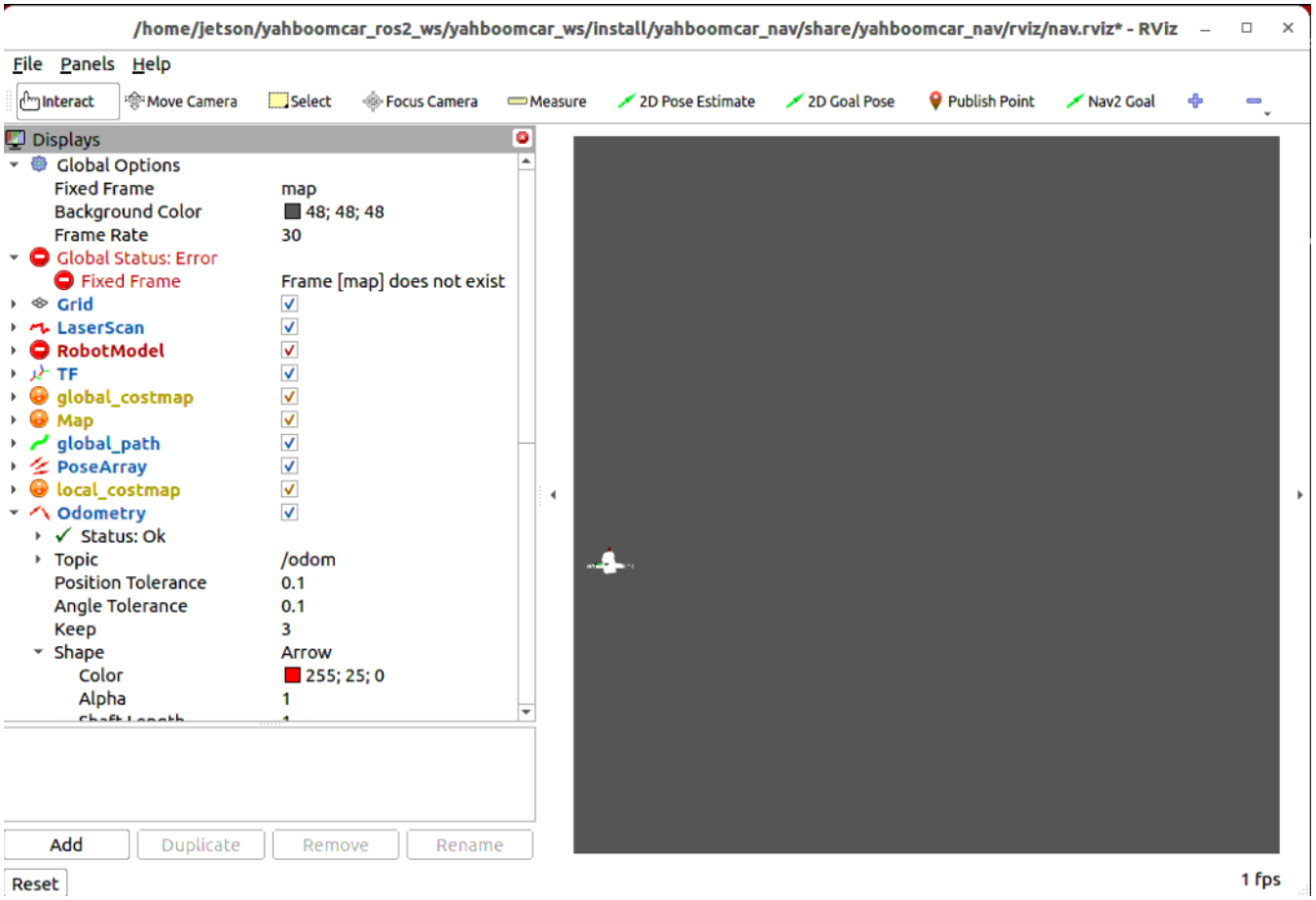
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



3.2 Start RViz Navigation Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

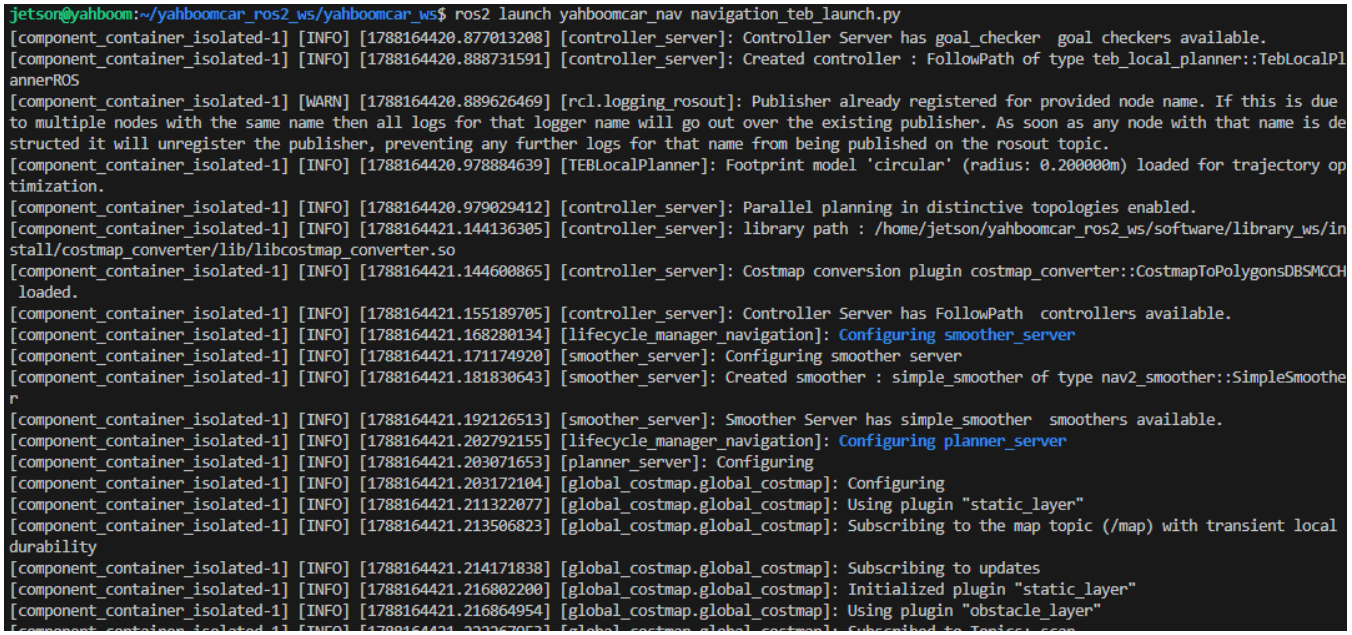
```
ros2 launch yahboomcar_nav display_nav_launch.py
```



3.3 Start Nav2 TEB Navigation Core

Run in **Terminal 3** (loads default `maps/yahboomcar.yaml` map and `teb_nav_params.yaml` parameters):

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

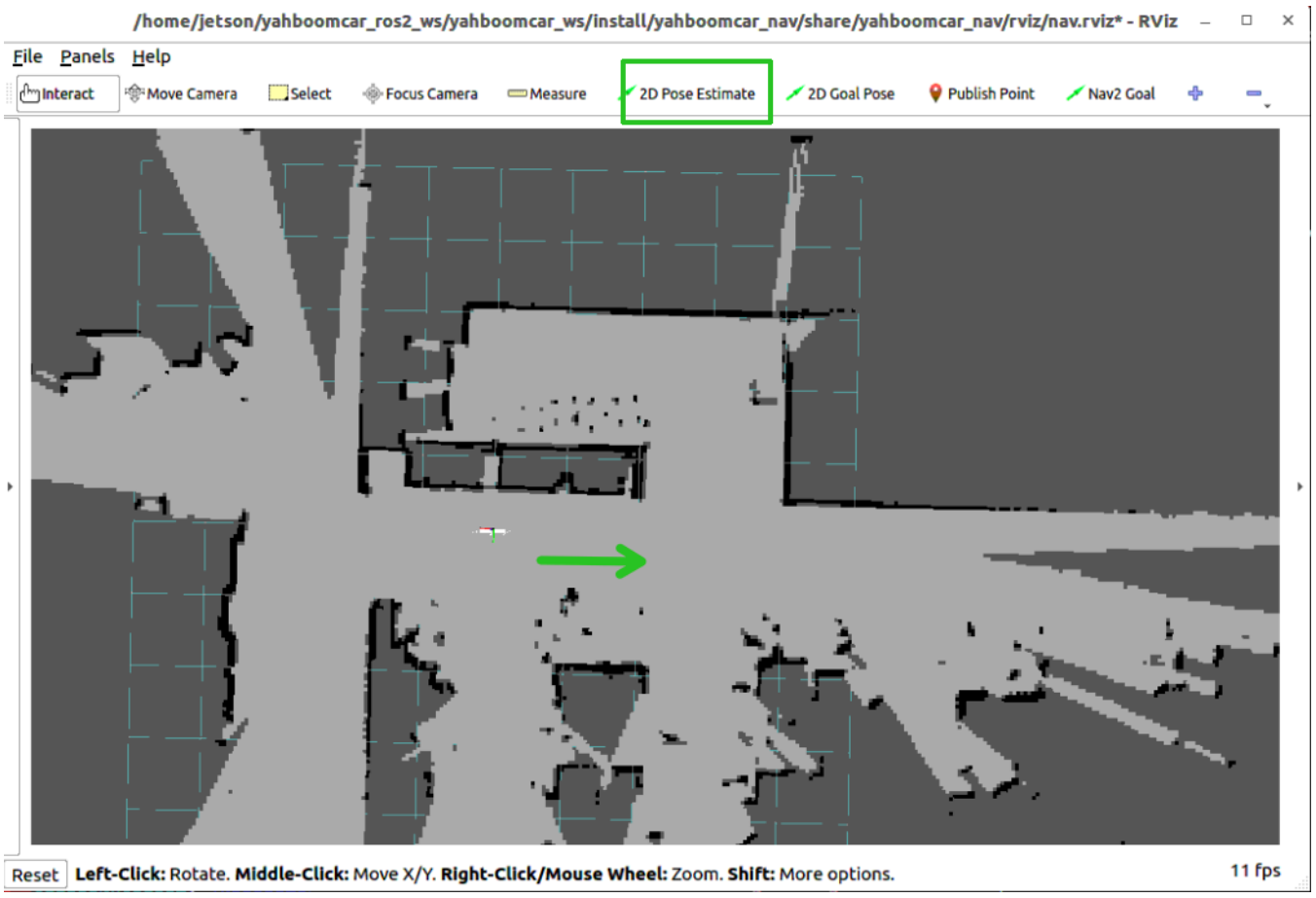


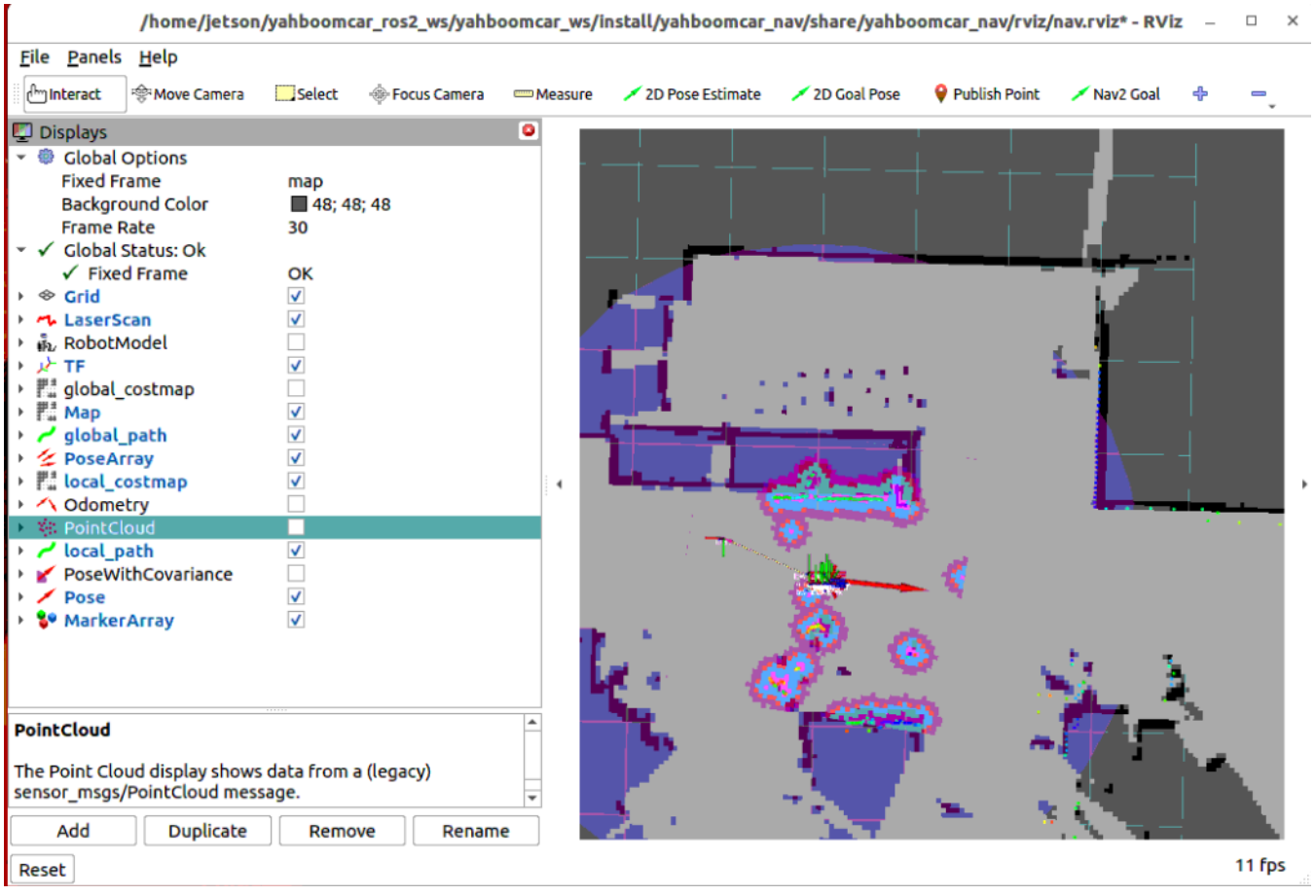
> **Tip**
>
> To load other custom maps, you can specify the map path through `map` parameter:
>
> ```
> ros2 launch yahboomcar_nav navigation_teb_launch.py
> map:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
> ```

4. Single Point Navigation Operation Flow and Effect Description

4.1 Set Initial Pose (2D Pose Estimate)

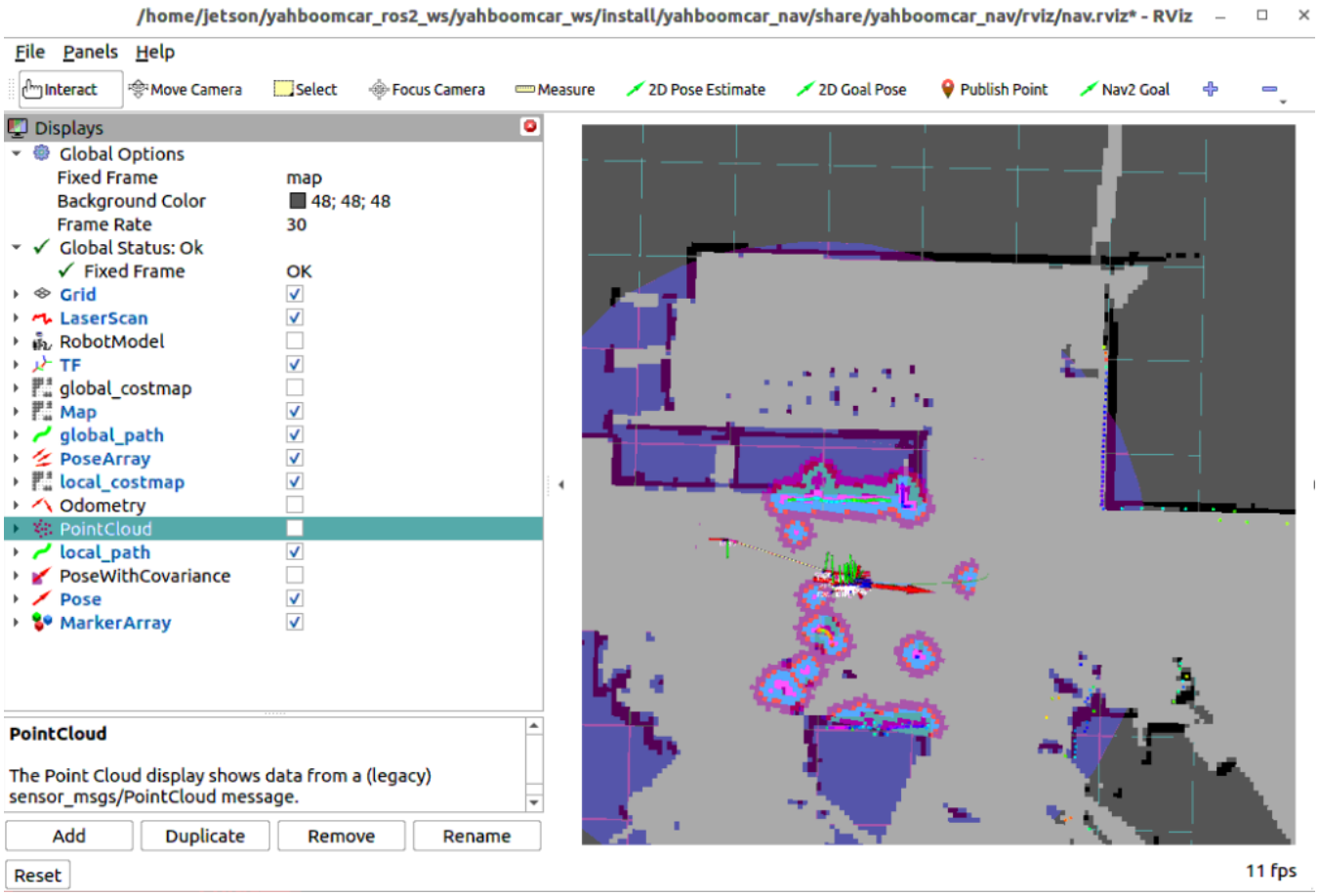
1. Click the `2D Pose Estimate` button in the top toolbar of RViz2.
2. Click at the actual physical position of the robot on the map and hold the mouse, drag out the green arrow along the actual chassis heading direction and release.
3. Observe that the green laser point cloud in RViz coincides with the black obstacle boundaries on the map, representing successful initial localization alignment.

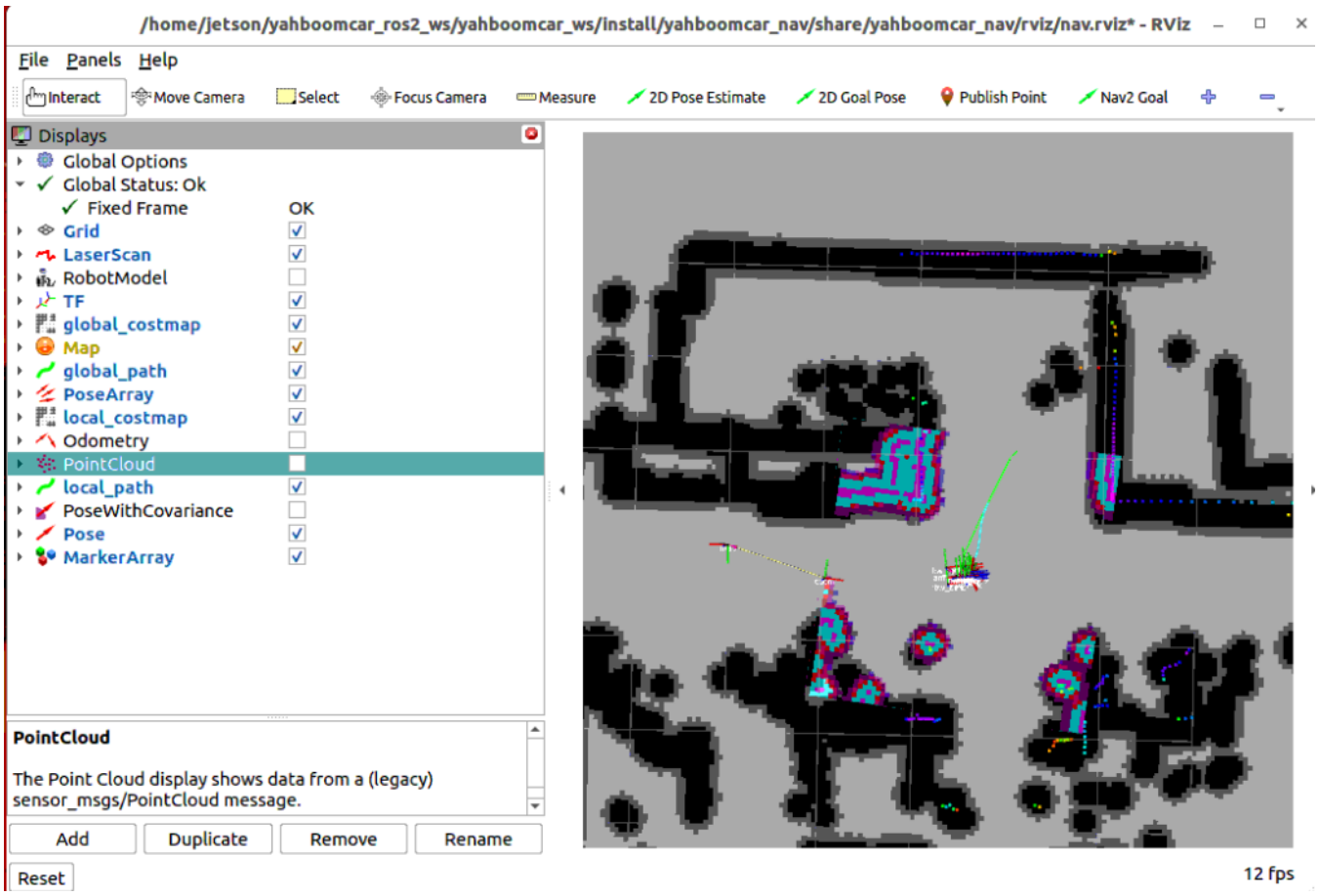

/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/nav.rviz* - RViz
File Panels Help
Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Goal Pose Publish Point Nav2 Goal
Reset Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel: Zoom. Shift: More options.
11 fps


/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/nav.rviz* - RViz
File Panels Help
Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Goal Pose Publish Point Nav2 Goal
Displays
Global Options
Fixed Frame map
Background Color 48; 48; 48
Frame Rate 30
Global Status: Ok
Fixed Frame OK
Grid
LaserScan
RobotModel
TF
global_costmap
Map
global_path
PoseArray
local_costmap
Odometry
PointCloud
local_path
PoseWithCovariance
Pose
MarkerArray
PointCloud
The Point Cloud display shows data from a (legacy) sensor_msgs/PointCloud message.
Add Duplicate Remove Rename
Reset
11 fps

4.2 Send Navigation Goal (2D Goal Pose)

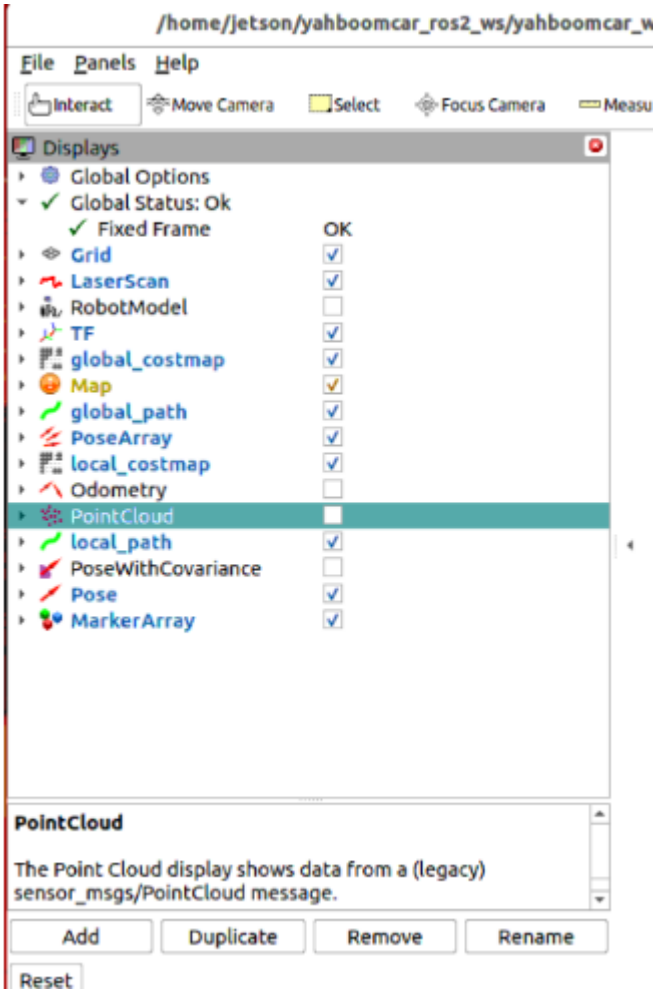
1. Click the `2D Goal Pose` (or Nav2 Goal) button in the top toolbar of RViz2.
2. Click in the traversable free area (gray area) on the map and drag out the expected goal heading arrow.
3. **Global Planning and Local Tracking**: The system will immediately generate a global planned path in the map, the robot autonomously starts front-wheel steering and smooth forward movement, dynamically bypasses sudden obstacles and drives toward the destination.




/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/nav.rviz* - RViz
File
Panels
Help
Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Goal Pose
Publish Point
Nav2 Goal
Displays
Global Options
Global Status: Ok
Fixed Frame
OK
Grid
LaserScan
RobotModel
TF
global_costmap
Map
global_path
PoseArray
local_costmap
Odometry
PointCloud
local_path
PoseWithCovariance
Pose
MarkerArray
PointCloud
The Point Cloud display shows data from a (legacy) sensor_msgs/PointCloud message.
Add
Duplicate
Remove
Rename
Reset
12 fps

Tip

In the Views panel on the right side of RViz, you can switch views and adjust observation height:



4.3 Goal Arrival and Terminal Feedback

When the robot drives to the goal and the position/angle error falls within the allowed tolerance range, the robot automatically stops smoothly, and the terminal will output goal completion prompt:

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_nav navigation_teb_launch.py
[component_container_isolated-1] [INFO] [1788164596.750350440] [amcl]: Setting pose (1788164596.750349): -0.899 0.471 -3.136
[component_container_isolated-1] [INFO] [1788164602.513565149] [amcl]: initialPoseReceived
[component_container_isolated-1] [WARN] [1788164602.513764443] [amcl]: Failed to transform initial pose in time (Lookup would require extrapolation into the future.  Requested time 1788164602.513669 but the latest data is at time 1788164602.445765, when looking up transform from frame [base_footprint] to frame [odom])
[component_container_isolated-1] [INFO] [1788164602.513808251] [amcl]: Setting pose (1788164602.513806): -1.039 0.431 -3.120
[component_container_isolated-1] [INFO] [1788164631.748109515] [bt_navigator]: Begin navigating from current location (-1.03, 0.43) to (-2.78, 0.30)
[component_container_isolated-1] [WARN] [1788164631.776650838] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164631.787204308] [controller_server]: Received a goal, begin computing control effort.
[component_container_isolated-1] [WARN] [1788164631.787406130] [controller_server]: No goal checker was specified in parameter 'current_goal_checker'. Server will use only plugin loaded goal_checker . This warning will appear once.
[component_container_isolated-1] [WARN] [1788164632.819800413] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164632.887584948] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [WARN] [1788164633.039198720] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164633.887570671] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [WARN] [1788164634.296688023] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164634.937573752] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [WARN] [1788164635.031450073] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [WARN] [1788164635.507653642] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [WARN] [1788164635.766819523] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164635.987569094] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [WARN] [1788164636.027518516] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [WARN] [1788164636.277634708] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164636.987651494] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [INFO] [1788164638.037658334] [controller_server]: Passing new path to controller.
[component_container_isolated-1] [WARN] [1788164638.267540277] [BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1788164638.296796200] [controller_server]: Reached the goal!
[component_container_isolated-1] [INFO] [1788164638.327585139] [bt_navigator]: Goal succeeded
```

> **Note**

> To end navigation, press `Ctrl+C` in each terminal window to stop the program.

5. Core Source Code and Navigation Parameter Analysis

5.1 Navigation Main Launch File Analysis

Launch File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/nav2/navigation_teb_launch.py
```

```
def generate_launch_description():
    package_path = get_package_share_directory('yahboomcar_nav')
    nav2_bringup_dir = get_package_share_directory('nav2_bringup')

    use_sim_time = LaunchConfiguration('use_sim_time', default='false')
    map_yaml_path = LaunchConfiguration(
        'map', default=os.path.join(package_path, 'maps', 'yahboomcar.yaml'))
    nav2_param_path = LaunchConfiguration(
        'params_file', default=os.path.join(package_path, 'params', 'teb_nav_params.yaml'))

    return LaunchDescription([
        DeclareLaunchArgument('use_sim_time', default_value=use_sim_time,
                              description='Use simulation clock if true'),
        DeclareLaunchArgument('map', default_value=map_yaml_path,
                              description='Full path to map file to load'),
        DeclareLaunchArgument('params_file', default_value=nav2_param_path,
                              description='Full path to param file to load'),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                [nav2_bringup_dir, '/launch', '/bringup_launch.py']),
            launch_arguments={
                'map': map_yaml_path,
                'use_sim_time': use_sim_time,
                'params_file': nav2_param_path}.items(),
        ),
    ])
```

- This Launch automatically calls official `nav2_bringup` framework, one-click pulling up Map Server, AMCL localization, Planner Server, Controller Server (TEB plugin), and Behavior Tree Navigator.

5.2 Core Algorithm Parameter Interpretation

Parameter Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/teb_nav_params.yaml
```

Core Parameter Description Table

Parameter Module	Parameter Name	Current Config Value	Core Function and Physical Meaning
TEB Speed Control	`max_vel_x`	`0.25`	Maximum allowed forward linear velocity during navigation (m/s)
TEB Speed Control	`min_vel_x`	`-0.05`	Strictly limit reverse speed (m/s, because no lidar at rear, low-speed reverse prevents collision)
TEB Steering Control	`max_vel_theta`	`1.0`	Maximum allowed turning angular velocity during navigation (rad/s, avoid rapid turning causing localization step loss)
TEB Dynamics	`acc_lim_x` / `acc_lim_theta`	`1.0` / `1.0`	Linear and angular acceleration limits (smooth acceleration/deceleration to prevent slipping)
Arrival Tolerance	`xy_goal_tolerance`	`0.25`	Goal arrival distance tolerance (meters)
Arrival Tolerance	`yaw_goal_tolerance`	`0.25`	Goal arrival heading angle tolerance (rad ≈ 14.3°)
Obstacle Avoidance	`min_obstacle_dist`	`0.10`	Minimum hard safety distance to maintain from obstacles (meters)
Obstacle Avoidance	`inflation_dist`	`0.10`	Local costmap obstacle soft inflation distance (meters)
AMCL Localization	`max_particles` / `min_particles`	`5000` / `1500`	Particle filter particle count (high particle count ensures front lidar localization convergence)
AMCL Localization	`laser_likelihood_max_dist`	`3.0`	Laser likelihood field matching maximum distance (meters)
AMCL Localization	`recovery_alpha_fast`	`0.1`	Fast recovery coefficient (automatically inject random particles for relocalization when pose is lost)

TEB and AMCL Core Parameter Configuration Snippet

```yaml
# AMCL adaptive particle filter localization configuration
amcl:
  ros__parameters:
    use_sim_time: False
    base_frame_id: "base_footprint"
    global_frame_id: "map"
```

```
    odom_frame_id: "odom"
    scan_topic: scan
    max_particles: 5000            # Maximum particle count
    min_particles: 1500            # Minimum particle count
    laser_likelihood_max_dist: 3.0 # Expanded matching distance to compensate for front
lidar FOV
    recovery_alpha_fast: 0.1       # Enable fast pose recovery mechanism
    update_min_a: 0.1              # Trigger localization update even for small rotation
angles
    update_min_d: 0.1              # Trigger localization update even for small
displacements

# TEB local path planner configuration
controller_server:
  ros__parameters:
    controller_frequency: 20.0
    FollowPath:
      plugin: "teb_local_planner::TebLocalPlannerROS"
      max_vel_x: 0.25              # Maximum cruising speed (m/s)
      min_vel_x: -0.05             # Strictly limit reverse speed (m/s)
      max_vel_theta: 1.0           # Maximum turning angular velocity (rad/s)
      acc_lim_x: 1.0               # Linear acceleration limit (m/s²)
      acc_lim_theta: 1.0           # Angular acceleration limit (rad/s²)
      min_obstacle_dist: 0.1       # Minimum obstacle safe spacing (m)
      inflation_dist: 0.1          # Obstacle avoidance inflation spacing (m)
      xy_goal_tolerance: 0.25      # Position arrival accuracy (m)
      yaw_goal_tolerance: 0.25     # Heading arrival accuracy (rad)

# Local costmap configuration
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 10.0
      publish_frequency: 5.0
      global_frame: odom
      robot_base_frame: base_footprint
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05
      plugins: ["obstacle_layer", "inflation_layer"]
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 10.0
        inflation_radius: 0.18     # Inflation radius (m)
```

5.3 Navigation Tuning Guide

1. **Adjust Cruising Speed and Steering Stability**:
   If you need to adjust the robot's straight-line navigation speed, modify `FollowPath.max_vel_x` (recommended `0.20 ~ 0.35 m/s`); if the chassis turns too fast at corners and easily slips causing AMCL pose step loss, reduce `FollowPath.max_vel_theta`.
2. **Adjust Obstacle Avoidance Safety Margin**:
   If the robot has difficulty passing through narrow passages, appropriately reduce `inflation_layer.inflation_radius` (inflation radius); if you want the robot to maintain more generous clearance from obstacles, increase `inflation_radius` or increase `cost_scaling_factor`.
3. **Adjust Goal Arrival Judgement Accuracy**:
   If the robot repeatedly fine-tunes its heading near the destination and has difficulty completing the task, appropriately relax `xy_goal_tolerance` and `yaw_goal_tolerance`.